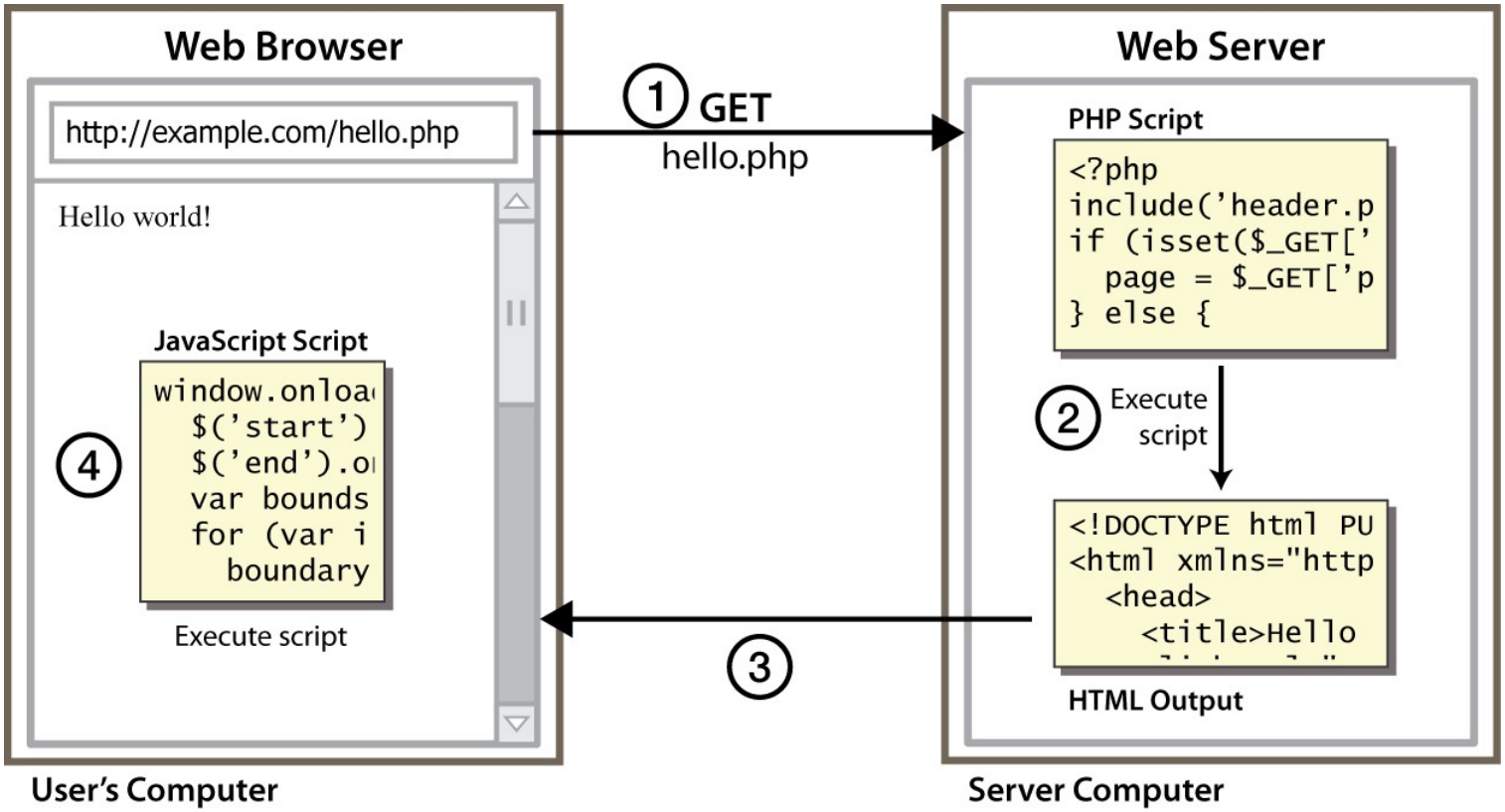
Introduction to JavaScript

Dr. Ahmad Mustafa
Department of software Engineering
University of Sargodha

Client Side Scripting



What is Javascript?

- a lightweight programming language ("scripting language")
  - used to make web pages interactive
  - insert dynamic text into HTML (ex: user name)
  - **react to events** (ex: page load user click)
  - get information about a user's computer (ex: browser type)
  - perform calculations on user's computer (ex: form validation)

What is JavaScript?

JavaScript is a **client-side scripting language** used to make web pages interactive.

It works together with:

- **HTML** → Structure
- **CSS** → Styling
- **JavaScript** → Behavior & Logic

Where JavaScript Runs?

- In browsers like Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Also runs on server-side using:

- Node.js

What is JavaScript?

Origin

Originally named "LiveScript," it was renamed JavaScript in 1995. This rebranding helped boost its popularity.

ECMAScript

The ECMAScript standard ensures cross-browser compatibility. This ensures code runs smoothly everywhere.

Interpreted

JavaScript is an interpreted language. Code is executed directly, without compilation.

Prototype-based

It uses prototypes for inheritance. This differs from class-based inheritance.

History of JavaScript

1

1995

Created by Brendan Eich at Netscape in just 10 days. A rapid development for a powerful language.

2

Netscape Navigator 2.0

The first browser to support JavaScript. This marked the beginning of its widespread use.

3

Interactive Pages

Initially designed to make web pages more interactive. It transformed static pages into dynamic experiences.

4

Complex Development

Evolved into complex application development. It now powers entire web applications.

Javascript vs Java

- interpreted, not compiled
- more relaxed syntax and rules
  - fewer and "looser" data types
  - variables don't need to be declared
  - errors often silent (few exceptions)
- key construct is the function rather than the class
  - "first-class" functions are used in many situations
- contained within a web page and integrates with its HTML/CSS content

Core Features of JavaScript

1. ## Variables

   Store data like numbers, text, and objects. These hold values used throughout your code.

2. ## Data Types

   Includes string, number, boolean, null, undefined, symbol, and object. Each serves a different purpose.

3. ## Operators

   Perform calculations and comparisons (+, -, *, /, ==, !=). Essential for logic and math.

4. ## Control Flow

   Use if/else statements and loops (for, while) for decision making. Dictate the order of execution.

JavaScript in Web Development

DOM Manipulation

Manipulate the DOM (Document Object Model) to change webpage structure.

User Events

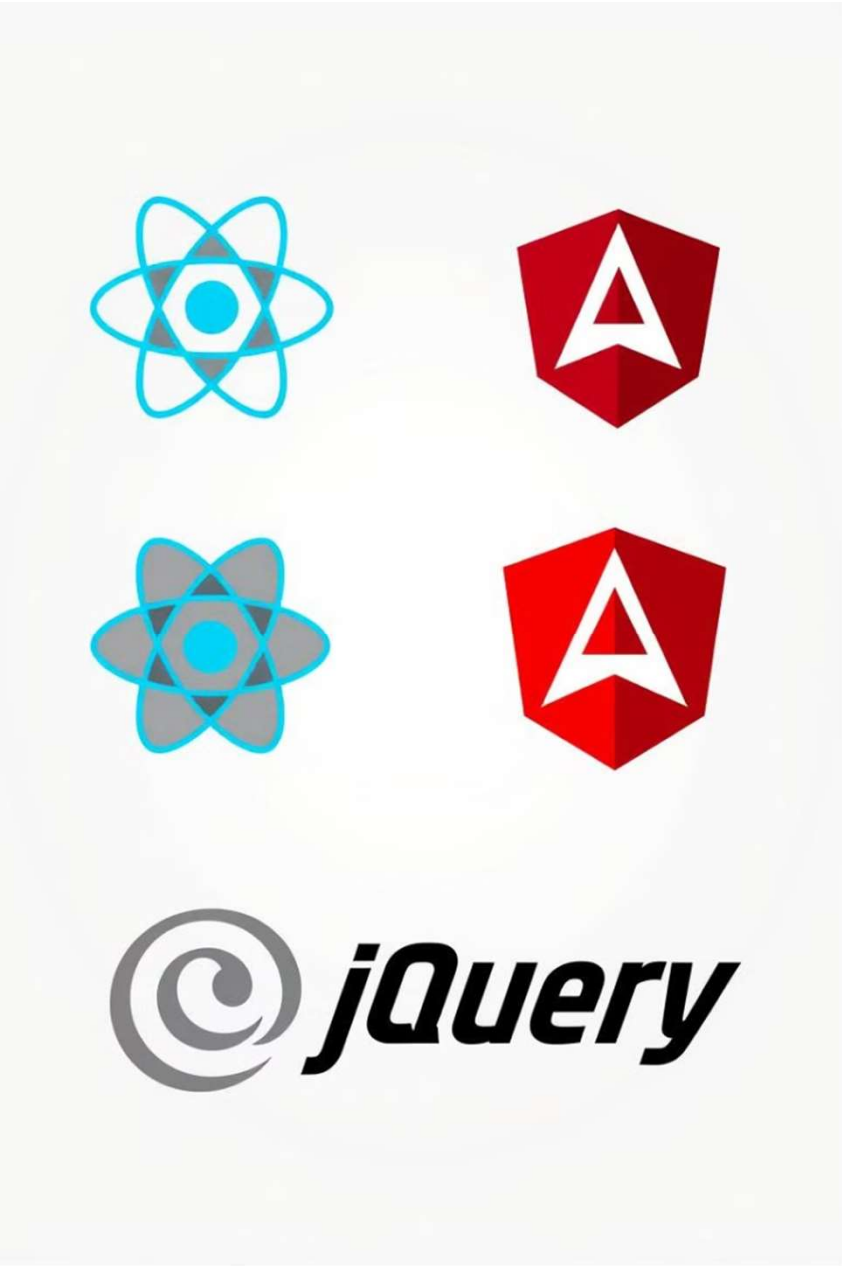
Respond to user events (clicks, mouseovers, form submissions). Create interactive experiences.

AJAX

Use AJAX (Asynchronous JavaScript and XML) for dynamic content loading. Update pages without reloads.

Interactive UIs

Creating interactive user interfaces for better UX.



JavaScript Frameworks and Libraries

React

A UI library by Facebook. Uses a component-based architecture, powering sites like Netflix and Instagram.

Angular

A comprehensive framework by Google. TypeScript-based, and used in Gmail and Forbes.

Vue.js

A progressive framework, easy to learn. Used by Alibaba and Xiaomi.

jQuery

Simplifies DOM manipulation and AJAX calls. A classic library for web development.

JavaScript Beyond the Browser

1

Node.js

Run JavaScript on the server-side. Enables back-end development with JavaScript.

2

APIs

Build APIs (Application Programming Interfaces). Connect different software systems.

3

Electron

Develop desktop applications. VS Code and Slack are built with Electron.

4

React Native

Create mobile apps. Build cross-platform applications.

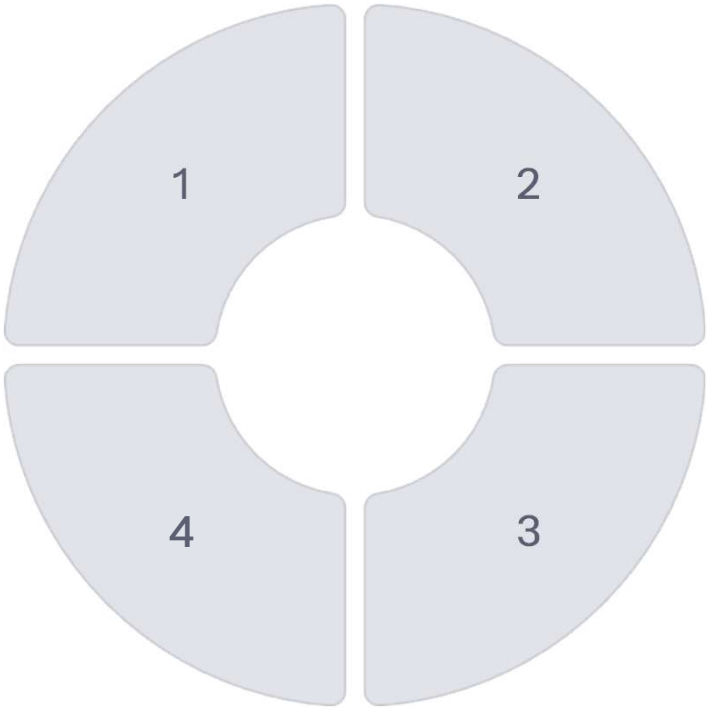
Benefits of Learning JavaScript

High Demand

A large job market for JavaScript developers. Many companies need JavaScript skills.

Continuous Evolution

Constantly updated with new features and best practices. Stay current with the latest trends.



Versatility

Used in front-end, back-end, and mobile development. A wide range of applications.

Large Community

Extensive resources, libraries, and support. Get help when you need it.

Getting Started with JavaScript

1 Developer Console

2 Text Editor

3 Online Resources

Start with your web browser's developer console to experiment. Use a text editor + browser to write and run JavaScript files. Use online resources to learn. Practice, practice, practice by building small projects.

Variables and Data Types

```
var name = "Ali";     // Old way
let age = 20;         // Block-scoped
const pi = 3.14;      // Cannot be reassigned
```

Keyword	Scope	Reassign	Redeclare
var	Function	Yes	Yes
let	Block	Yes	No
const	Block	No	No

Example

- // var example
- var x = 10;
- var x = 20; // Re-declaration allowed
- x = 30; // Update allowed
- console.log(x); // Output: 30
- // let example
- let y = 10;
- // let y = 20; // Re-declaration NOT allowed
- y = 25; // Update allowed
- console.log(y); // Output: 25
- // const example
- const z = 10;
- // z = 20; // Re-assignment NOT allowed
- console.log(z); // Output: 10

- // Block scope demonstration
- if (true) {
- var a = 1;
- let b = 2;
- const c = 3;}
- console.log(a); // Works (var is function/global scoped)
- // console.log(b); // Error (let is block scoped)
- // console.log(c); // Error (const is block scoped)

Data Types

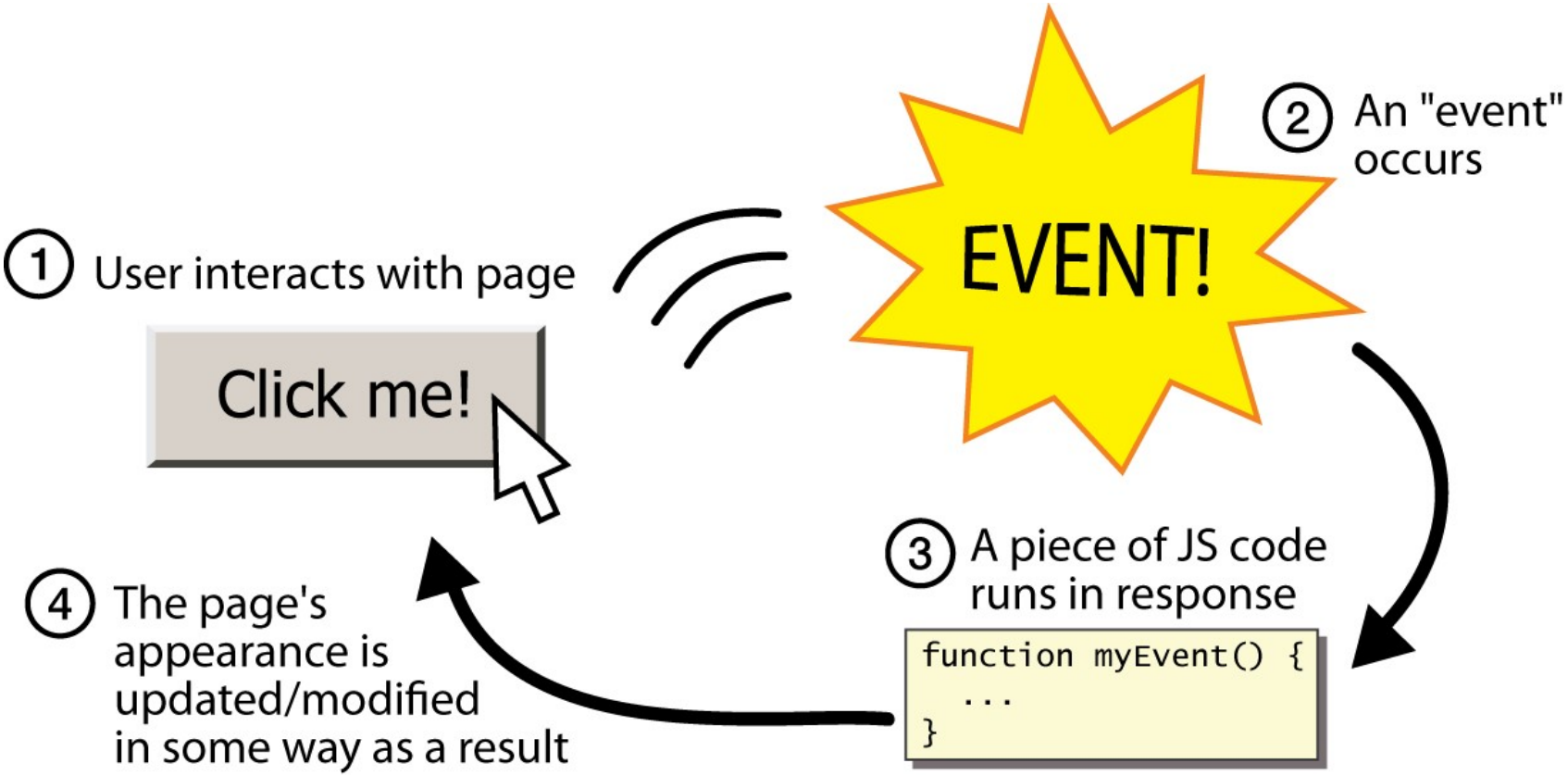
- **Primitive Data Types**
- let name = "Ahmad"; // String
  let age = 25; // Number
  let isActive = true; // Boolean
  let x = null; // Null
  let y; // Undefined
- **Non-Primitive**
- let student = {name: "Ali", age: 22}; // Object
  let marks = [80, 90, 75]; // Array

Linking to a JavaScript file: `script`

```
<script src="filename" type="text/javascript"></script>
```
HTML

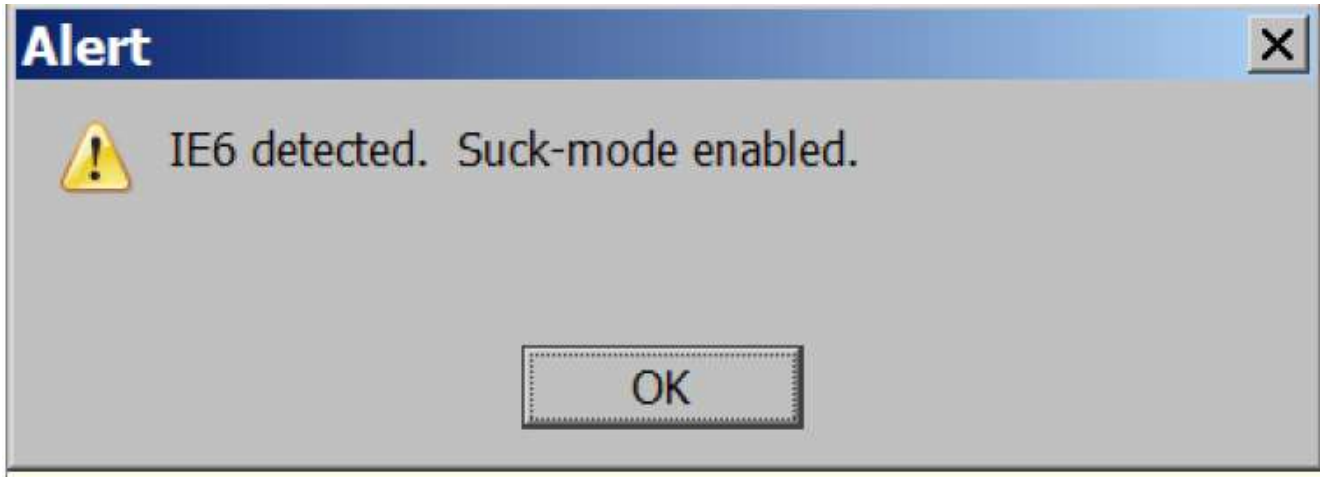
- script tag should be placed in HTML page's head
- script code is stored in a separate .js file
- JS code can be placed directly in the HTML file's body or head (like CSS)
  - but this is bad style (should separate content, presentation, and behavior

Event-driven programming



A JavaScript statement: `alert`

```
alert("IE6 detected. Suck-mode enabled.");
                                          JS
```



- a JS command that pops up a dialog box with a message

Buttons

```
<button>Click me!</button>
```
HTML

- button's text appears inside tag; can also contain images
- To make a responsive button or other UI control:
  1. choose the control (e.g. button) and event (e.g. mouse 1. click) of interest
  2. write a JavaScript function to run when the event occurs
  3. attach the function to the event on the control

JavaScript functions

```
function name() {
statement ;
statement ;
...
statement ;
}
```
JS

```
function myFunction() {
      alert("Hello!");
      alert("How are you?");
}
```
JS

- the above could be the contents of example.js linked to our HTML page
- statements placed into functions can be evaluated in response to user events

Event handlers

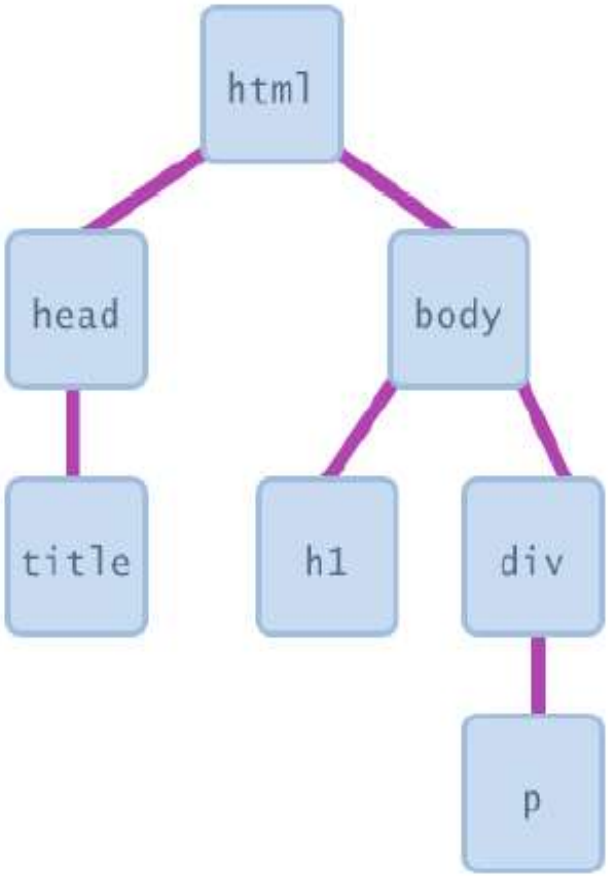
```
<element attributes onclick="function();">...
```
HTML

```
<button onclick="myFunction();">Click me!</button>
```
HTML

- JavaScript functions can be set as event handlers
  - when you interact with the element, the function will execute
- onclick is just one of many event HTML attributes we'll use
- but popping up an alert window is disruptive and annoying
  - A better user experience would be to have the message appear on the page...

Document Object Model (DOM)

- most JS code manipulates elements on an HTML page
- we can examine elements' state
  - e.g. see whether a box is checked
- we can change state
  - e.g. insert some new text intc a div
- we can change styles
  - e.g. make a paragraph red



DOM element objects

HTML

```
<p>
  Look at this octopus:
  <img src="octopus.jpg" alt="an octopus" id="icon01" />
  Cute, huh?
</p>
```

DOM Element Object

Property	Value
tagName	"IMG"
src	"octopus.jpg"
alt	"an octopus"
id	"icon01"

JavaScript

```
var icon = document.getElementById("icon01");
icon.src = "kitty.gif";
```

Accessing elements: `document.getElementById`

```
var name = document.getElementById("id");
```
JS

```
<button onclick="changeText();">Click me!</button>
<span id="output">replace me</span>
<input id="textbox" type="text" />
```
HTML

```
function changeText() {
      var span = document.getElementById("output");
      var textBox = document.getElementById("textbox");

       textbox.style.color = "red";

}
```
JS

Accessing elements:

`document.getElementById`

- document.getElementById returns the DOM object for an element with a given id
- can change the text inside most elements by setting the innerHTML property
- can change the text in form controls by setting the value property

Changing element style: `element.style`

Attribute	Property or style object
color	color
padding	padding
background-color	backgroundColor
border-top-width	borderTopWidth
Font size	fontSize
Font famiy	fontFamily

Preetify

```
function changeText() {
      //grab or initialize text here

      // font styles added by JS:
      text.style.fontSize = "13pt";
      text.style.fontFamily = "Comic Sans MS";
      text.style.color = "red"; // or pink?
}
```
JS

More Javascript Syntax

Variables

```
var name = expression;                                    JS
```

```
var clientName = "Connie Client";
var age = 32;
var weight = 127.4;                                       JS
```

- variables are declared with the var keyword (case sensitive)
- types are not specified, but JS does have types ("loosely typed")
  - `Number, Boolean, String, Array, Object, Function, Null, Undefined`
  - can find out a variable's type by calling `typeof`

Number type

```
var enrollment = 99;
var medianGrade = 2.8;
var credits = 5 + 4 + (2 * 3);
```

JS

- integers and real numbers are the same type (no int vs. double)
- same operators: + - * / % ++ -- = += -= *= /= %=
- similar precedence to Java
- many operators auto-convert types: "2" * 3 is 6

Comments (same as Java)

```
// single-line comment
/* multi-line comment */
```

JS

- identical to Java's comment syntax
- recall: 4 comment syntaxes
  - HTML: <!-- comment -->
  - CSS/JS/PHP: /* comment */
  - Java/JS/PHP: // comment
  - PHP: # comment

Math object

```
var rand1to10 = Math.floor(Math.random() * 10 + 1);
var three = Math.floor(Math.PI);
```
JS

- methods: `abs, ceil, cos, floor, log, max, min, pow, random, round, sin, sqrt, tan`
- properties: `E, PI`

Special values: null and undefined

```
var ned = null;
var benson = 9;
// at this point in the code,
// ned is null
// benson's 9
// caroline is undefined
```
JS

- `undefined` : has not been declared, does not exist
- `null` : exists, but was specifically assigned an empty or null value
- Why does JavaScript have both of these?

Logical operators

- > < >= <= && || ! == != === !==
- most logical operators automatically convert types:
  - 5 < "7" is true
  - 42 == 42.0 is true
  - "5.0" == 5 is true
- === and !== are strict equality tests; checks both type and value
  - "5.0" === 5 is false

if/else statement (same as Java)

```
if (condition) {
      statements;
} else if (condition) {
      statements;
} else {
      statements;
}
```

JS

- identical structure to Java's if/else statement
- JavaScript allows almost anything as a condition

Boolean type

```
var iLike190M = true;
var ieIsGood = "IE6" > 0; // false
if ("web devevelopment is great") { /* true */ }
if (0) { /* false */ }
```
JS

- any value can be used as a Boolean
  - "falsey" values: 0, 0.0, NaN, "", null, and undefined
  - "truthy" values: anything else
- converting a value into a Boolean explicitly:
  - `var boolValue = Boolean(otherValue);`
  - `var boolValue = !!(otherValue);`

for loop (same as Java)

```
var sum = 0;
for (var i = 0; i < 100; i++) {
      sum = sum + i;
}
```
JS

```
var s1 = "hello";
var s2 = "";
for (var i = 0; i < s.length; i++) {
      s2 += s1.charAt(i) + s1.charAt(i);
}
// s2 stores "hheelllloo"
```
JS

while loops (same as Java)

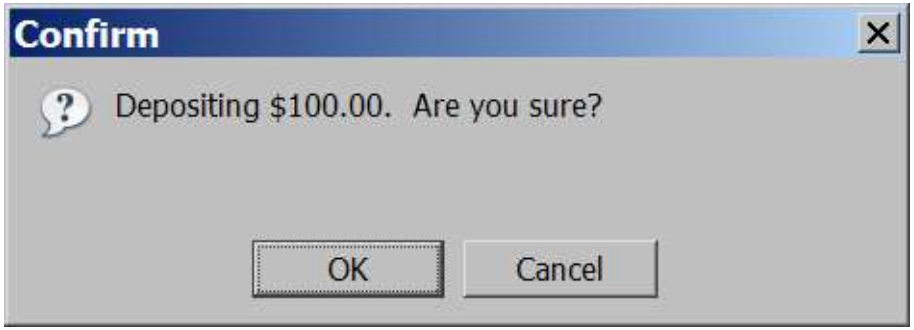
```
while (condition) {
      statements;
}
```
JS

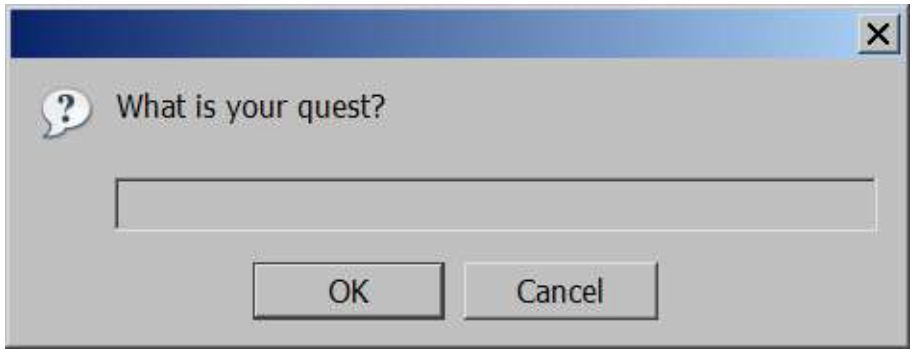
```
do {
   statements;
} while (condition);
```
JS

- break and continue keywords also behave as in Java

Popup boxes

```
alert("message"); // message
confirm("message"); // returns true or false
prompt("message"); // returns user input string
                                          JS
```





Arrays

```
var name = []; // empty array
var name = [value, value, ..., value]; // pre-filled
name[index] = value; // store element
```
JS

```
var ducks = ["Huey", "Dewey", "Louie"];
var stooges = []; // stooges.length is 0
stooges[0] = "Larry"; // stooges.length is 1
stooges[1] = "Moe"; // stooges.length is 2
stooges[4] = "Curly"; // stooges.length is 5
stooges[4] = "Shemp"; // stooges.length is 5
```
JS

Array methods

```
var a = ["Stef", "Jason"]; // Stef, Jason
a.push("Brian"); // Stef, Jason, Brian
a.unshift("Kelly"); // Kelly, Stef, Jason, Brian
a.pop(); // Kelly, Stef, Jason
a.shift(); // Stef, Jason
a.sort(); // Jason, Stef
```

JS

- array serves as many data structures: list, queue, stack, ...
- methods: `concat, join, pop, push, reverse, shift, slice, sort, splice, toString, unshift`
  - push and pop add / remove from back
  - unshift and shift add / remove from front
  - shift and pop return the element that is removed

`String` type

```
var s = "Connie Client";
var fName = s.substring(0, s.indexOf(" ")); // "Connie"
var len = s.length; // 13
var s2 = 'Melvin Merchant';
```

JS

- **Methods:** charAt, charCodeAt, fromCharCode, indexOf, lastIndexOf, replace, split, substring, toLowerCase, toUpperCase
  - charAt returns a one-letter String (there is no char type)
- length property (not a method as in Java)
- Strings can be specified with "" or ''
- concatenation with + :
  - 1 + 1 is 2, but "1" + 1 is "11"

More about `String`

- escape sequences behave as in Java: \' \" \& \n \t \\
- converting between numbers and Strings:

```
var count = 10;
var s1 = "" + count; // "10"
var s2 = count + " bananas, ah ah ah!"; // "10 bananas, ah ah ah!"
var n1 = parseInt("42 is the answer"); // 42
var n2 = parseFloat("booyah"); // NaN
```
JS

- accessing the letters of a String:

```
var firstLetter = s[0]; // fails in IE
var firstLetter = s.charAt(0); // does work in IE
var lastLetter = s.charAt(s.length - 1);
```
JS

Splitting strings: split and join

```
var s = "the quick brown fox";
var a = s.split(" "); // ["the", "quick", "brown", "fox"]
a.reverse(); // ["fox", "brown", "quick", "the"]
s = a.join("!"); // "fox!brown!quick!the"
```
JS

- split breaks apart a string into an array using a delimiter
  - can also be used with regular expressions (seen later)
- join merges an array into a single string, placing a delimiter between them

Conclusion: The Power of JavaScript

JavaScript is essential for modern web development. It's a versatile language with broad applications. Continuous learning and growth are key to mastering it. Thank you!